MASTER QA LEARNING PROGRAM

Phase 2: Detailed Learning Guide — Level 1

Junior QA Engineer: Complete Topic Mastery

8 Chapters | 8–10 Weeks | Foundation Level

(Update this Table of Contents in Word: References > Update Table)

Learning Objectives

1. Master the Software Development Life Cycle (SDLC) and understand QA's role in every phase.
2. Apply the Software Testing Life Cycle (STLC) to plan, execute, and close testing activities professionally.
3. Manage defects through the full defect life cycle using industry-standard tools and practices.
4. Select and apply the correct testing type for any given test scenario or project context.
5. Design high-quality test cases using BVA, EP, Decision Tables, and State Transition techniques.
6. Perform requirement analysis and build a complete Requirement Traceability Matrix (RTM).
7. Write professional defect reports and conduct effective defect triage sessions.
8. Execute tests, track metrics, and produce sprint-level test reports for stakeholders.

Phase 2 Scope

This document provides the full concept-to-mastery treatment for every Level 1 topic. Each chapter includes concept explanation, business importance, industry best practices, real-world examples, templates, interview questions, scenario-based exercises, and case studies.

Chapter 1 - SDLC: From Requirements to Release

↑ Back to Index

The Software Development Life Cycle is the backbone of every software project. It is the structured process that takes a business idea from concept through design, development, testing, deployment, and maintenance. For a QA professional, the SDLC is not just background knowledge — it is the map that tells you when to start testing, what to test, how much time you have, and what evidence of quality to produce at each stage.

Many junior testers make the mistake of thinking QA starts when developers hand over a build. In reality, quality work begins at the requirements phase and never stops. Understanding the SDLC means you can anticipate what is coming, prepare in advance, and catch problems at the cheapest possible stage — before code is even written.

1.1 SDLC Models Explained

Different organisations and projects use different SDLC models. Each model determines how development and testing activities are sequenced. As a QA professional you must understand the model your project follows so you can plan testing activities accordingly.

SDLC Model	Description	Testing Approach	Best Suited For
Waterfall	Sequential phases: Requirements > Design > Dev > Test > Deploy. No going back.	Testing only starts after development is fully complete.	Fixed-scope projects, regulatory systems, government contracts
V-Model	Each development phase has a corresponding testing phase. Verification and validation in parallel.	Test planning starts at each dev phase. Execution at end.	Safety-critical systems, medical devices, avionics
Agile (Scrum)	Iterative 2-week sprints. Requirements evolve. Continuous delivery.	Testing in every sprint. QA embedded in team.	Web apps, mobile apps, product companies, startups
Kanban	Continuous flow of work items with WIP limits. No fixed sprint length.	Testing is a continuous activity. QA pulls work from backlog.	Maintenance projects, support teams, ops workflows
Spiral	Risk-driven iterative model. Each spiral adds functionality.	Risk-based testing at each spiral iteration.	Large high-risk projects with complex requirements
DevOps/CI-CD	Continuous integration and deployment. Code changes shipped multiple times per day.	Automated testing in pipeline. Shift-left and shift-right.	Cloud-native, SaaS, high-frequency release teams

How It Works Internally

In an Agile/Scrum project the SDLC compresses into a 2-week sprint. The product owner writes user stories (mini-requirements), the team plans sprint work, developers build features, QA tests them, and the sprint ends with a potentially releasable product. The SDLC phases still exist but they happen in miniature within each sprint. Understanding this helps you explain to business stakeholders why testing cannot start on Day 13 of a 14-day sprint.

1.2 QA Role in Each SDLC Phase

Phase	What Happens	QA Activities	QA Deliverables
Requirements	Business analysts document what the system must do.	Review requirements for completeness, clarity, testability. Raise queries. Identify test conditions.	Requirements Review Report, Initial Test Conditions List, RTM skeleton
System Design	Architects design the system structure, databases, APIs, integrations.	Review design documents. Identify integration test points. Flag untestable designs.	Design Review Checklist, Integration Test Approach
Detailed Design	Developers design individual modules and components.	Review unit test approach. Identify component-level test cases.	Component Test Cases, Test Data Requirements
Development/Coding	Developers write code. Unit tests written alongside.	Finalise test cases. Prepare test environment. Create test data. Review dev unit tests.	Test Cases (complete), Test Data Set, Test Environment Setup Report
Testing	QA executes tests. Defects found and fixed. Regression performed.	Execute functional, regression, integration, performance, security, UAT tests. Track and report progress.	Test Execution Reports, Defect Reports, Daily Status Updates, Test Summary Report
Deployment	Tested software deployed to production environment.	Smoke test in production. Monitor for deployment defects. Validate data migration.	Deployment Test Checklist, Smoke Test Report, Sign-off Document
Maintenance	Bug fixes, patches, and enhancements post-production.	Regression test patches. Impact analysis. Validate hot fixes in production.	Patch Test Reports, Regression Impact Analysis

1.3 Entry and Exit Criteria

Entry and exit criteria are the conditions that must be met before testing can begin (entry) and before testing is considered complete (exit). Defining these upfront prevents premature testing and provides clear completion milestones.

Criteria Type	Example Conditions
Entry Criteria (to start testing)	Requirements sign-off received. Development complete for the sprint or module. Test environment available and stable. Test cases reviewed and approved. Test data prepared.
Exit Criteria (to stop testing)	All planned test cases executed. Pass rate above agreed threshold (e.g. >95%). Zero P1/Critical open defects. All P2/High defects either resolved or deferred with mitigation. Test Summary Report approved by stakeholders.
Suspension Criteria (pause testing)	Test environment unavailable. More than 30% of test cases blocked by a single defect. Critical build instability preventing execution.
Resumption Criteria (restart testing)	Environment restored and validated. Blocking defect resolved. New build verified stable by smoke testing.

In Practice

In Agile projects entry and exit criteria are often embedded in the Definition of Ready (DoR) and Definition of Done (DoD). A story meets the DoR when it has clear acceptance criteria and is testable. A story meets the DoD when all acceptance criteria pass, defects are resolved or deferred, and test evidence is committed to the test management tool. Learn these definitions early — they are asked about in almost every QA interview.

1.4 Real-World Analogy

Think of the SDLC as building a house. The requirements phase is when the client tells the architect what they want — number of rooms, style, budget. Design is when the architect draws blueprints. Development is when the builders construct the house. Testing is when the building inspector checks every room, wire, and pipe against the approved plans. Deployment is when the client moves in. Maintenance is all the plumbing fixes and paint jobs that follow. Would you let the client move in before the building inspector has checked the wiring? That is what releasing software without QA looks like.

1.5 Worked Example: SDLC in an Agile Sprint

```
Sprint 14 - Feature: Customer Order Tracking

Day 1-2  (Requirements): QA reviews user stories, writes test conditions, creates
RTM
Day 1-3  (Design): QA reviews API design, identifies integration test points
Day 3-7  (Development): QA writes test cases, prepares test data, sets up test env
```

```
Day 7-12 (Testing): QA executes 45 test cases, finds 8 defects, 6 resolved
Day 12-13 (UAT): Business user validates 5 key scenarios
Day 14   (Deployment): Smoke test on staging, sign-off, release to prod

Entry Criteria Met: Dev sign-off Day 6, Env ready Day 6, Test Cases reviewed Day 5
Exit Criteria Met: 44/45 passed (97.8%), 0 P1 open, 1 P3 deferred
Sprint Outcome: RELEASED
```

1.6 Best Practices

- Get involved at requirements stage, not just testing stage. Every hour spent reviewing requirements saves 10 hours of defect fixing later.
- Always agree on entry and exit criteria in writing before testing begins. Verbal agreements are forgotten under release pressure.
- Understand which SDLC model your project uses on Day 1. It determines your entire testing calendar.
- In Agile, attend all ceremonies including backlog grooming. Stories are often changed before sprint planning and you need to know what is coming.
- Document every assumption you make during requirements review. Assumptions become the most common source of test gaps.
- Build a test environment checklist and validate it before every sprint. Environment issues are the number one cause of wasted testing time.

1.7 Common Mistakes

- Starting test case writing before requirements are stable: Requirements that change after test cases are written create rework. Wait for sign-off or mark test cases as draft until sign-off.
- Treating SDLC phases as waterfall in an Agile project: In Agile all phases overlap. You plan and test simultaneously. Do not wait for a handover.
- Skipping the design review: Design documents reveal testability issues early. A system designed without testability in mind becomes a nightmare to test.
- Not updating exit criteria when scope changes: If 10 stories are added mid-sprint, the exit criteria must be renegotiated. Do not silently accept extra scope.

Warning

Never start formal test execution without confirmed entry criteria. If the environment is not ready, test cases are not reviewed, or requirements are still changing, any testing you do will have to be repeated. Document that entry criteria are not met and formally request a delay rather than testing on an unstable baseline.

1.8 Exercises

1. Draw the Waterfall SDLC from memory. Annotate each phase with at least 3 QA activities and 2 QA deliverables.

2. Compare Waterfall and Agile SDLC models. Create a table listing 5 differences in how QA operates in each model.
3. For a project to build an online banking application, write complete entry and exit criteria for the Testing phase.
4. You are a junior QA engineer on an Agile project. Sprint planning is tomorrow. Write 5 questions you would ask the product owner about the upcoming user stories to ensure testability.
5. Research your company's or a public project's SDLC model. Write a one-page summary explaining which model it follows and how testing fits into it.

Quick Quiz

1. Name four common SDLC models and describe in one sentence how testing differs in each.
2. What is the difference between Entry Criteria and Exit Criteria? Why are both necessary?
3. In an Agile project, which SDLC phases happen within a single sprint?
4. What are Suspension Criteria and when would you invoke them?
5. You are asked to start testing but the test environment is not ready. What do you do and how do you document it?
6. What is the Definition of Done in Agile and how does it relate to QA exit criteria?

Interview Questions

1. Walk me through the SDLC and explain how your QA role changes at each phase.
2. What is the difference between the V-Model and Agile when it comes to testing?
3. How do you define and communicate entry and exit criteria to your team?
4. Describe a situation where you started testing before entry criteria were met. What happened and what did you learn?
5. How does the SDLC model affect how you plan and estimate testing effort?

Chapter Summary

• The SDLC defines how software is built. QA has a role in every single phase, not just the testing phase.
• Different SDLC models (Waterfall, Agile, V-Model, DevOps) require very different testing approaches and timelines.
• Entry and exit criteria are the QA professional's contract with the project. Define them in writing before testing starts.
• In Agile, the SDLC phases compress into each sprint. QA must be active from Day 1, not Day 10.
• Design reviews are part of QA. Catching untestable designs early prevents significant rework.
• The most expensive defects are those found in production. The SDLC is designed to catch them earlier.

Chapter 2 - STLC: The QA Process Framework

↑ Back to Index

While the SDLC governs how the entire software project is run, the Software Testing Life Cycle is the QA team's own internal process. It is the sequence of activities that ensures testing is planned, designed, executed, and closed in a disciplined, repeatable, and auditable way. Every professional QA engagement follows some form of the STLC, whether the team calls it that or not.

Understanding the STLC transforms you from someone who just executes tests into someone who manages a quality process. When a project manager asks 'how is testing going?' you should be able to point to exactly which STLC phase you are in, what has been completed, and what is outstanding.

2.1 The Six Phases of the STLC

Phase	Primary Activity	Key QA Deliverable	Duration (typical)
1. Requirement Analysis	Understand and analyse requirements for testability	Requirement Review Report, RTM skeleton	10-15% of test effort
2. Test Planning	Define test strategy, estimate effort, plan resources	Test Plan document	15-20% of test effort
3. Test Case Development	Write test cases, design test data, prepare scripts	Test Cases, Test Data, Automation Scripts	25-30% of test effort
4. Test Environment Setup	Configure and validate test environments and tools	Environment Setup Report, Tool Configuration	5-10% of test effort
5. Test Execution	Execute test cases, log defects, retest fixes, regress	Test Execution Report, Defect Reports	35-45% of test effort
6. Test Cycle Closure	Analyse results, prepare final report, capture lessons learned	Test Summary Report, Lessons Learned	5-10% of test effort

2.2 Phase 1 — Requirement Analysis

This is where quality begins. QA analyses the requirements document, user stories, or product backlog to understand what needs to be tested, identify ambiguities, and build the first version of the Requirement Traceability Matrix.

Activity	How to Perform It	Output
Review requirements	Read every requirement. Highlight unclear, missing, or conflicting items.	List of queries for the BA or Product Owner
Identify test conditions	For each requirement, identify 'what needs to be verified'. Each condition becomes a test case later.	Test Conditions List
Assess testability	Ask: can this requirement be tested? If not, why not? What would make it testable?	Testability Assessment Report
Create RTM skeleton	Map each requirement ID to placeholder test case IDs	Initial RTM (empty test case column)
Estimate test scope	Count requirements and estimate number of test cases per requirement	Test scope estimate for planning

2.3 Phase 2 — Test Planning

The Test Plan is the single most important QA document on a project. It is the master document that defines how testing will be conducted. A well-written Test Plan gives everyone — including clients and project managers — confidence that quality is being managed professionally.

Test Plan Section	Content	Example
Test Scope	What is and is not being tested	In Scope: Login, Registration, Profile. Out of Scope: Payment (Phase 2).
Test Strategy	Approach for each test type	Manual for functional. Automation for regression. JMeter for performance.
Test Objectives	What we aim to achieve	Validate all requirements. Achieve 95% pass rate. Zero P1 defects at release.
Resource Plan	Who is testing, their roles and responsibilities	2 QA Engineers, 1 Automation QA, 1 Performance QA.
Test Schedule	Timeline for each STLC phase	Test case writing: Week 1-2. Execution: Week 3-5. UAT: Week 6.
Risk & Mitigation	What could go wrong with testing	Risk: Late builds. Mitigation: Start API testing before UI is ready.
Entry/Exit Criteria	Start and stop conditions	Entry: Dev sign-off. Exit: 97% pass rate, 0 P1 open.

Test Plan Section	Content	Example
Defect Management	Tools, workflow, severity/priority definitions	Tool: JIRA. Triage: Daily 30-min meeting at 10am.

2.4 Phase 3 — Test Case Development

This is where test cases, test data, and automation scripts are created. Test cases must be complete, unambiguous, and independently executable. A test case written by one tester should produce the same result when executed by any other tester.

```
Standard Test Case Format:

Test Case ID  : TC-REG-001
Module        : User Registration
Requirement ID: REQ-005
Title         : Verify successful registration with valid data
Priority      : High
Preconditions : User is on the Registration page. Browser: Chrome 124.

Test Steps:
1. Enter valid first name: John
2. Enter valid last name: Doe
3. Enter valid email: john.doe@example.com
4. Enter password: Secure@123 (meets complexity rules)
5. Confirm password: Secure@123
6. Click Register button

Expected Result:
  - User is redirected to the Dashboard page
  - Welcome email sent to john.doe@example.com
  - User record created in the database
  - Session token generated and stored in cookie

Test Data: See TestData_Registration.xlsx, Row 1
Actual Result: [Filled during execution]
Status: [Pass/Fail/Blocked]
```

2.5 Phase 4 — Test Environment Setup

The test environment is the infrastructure on which testing is performed. It must closely mirror production to produce meaningful test results. Environment issues — wrong configuration, missing test data, connectivity problems — cause more test failures than actual application bugs.

Environment Component	What to Validate	Common Issues
Application	Correct build deployed. Version number verified.	Wrong build deployed. Missing environment-specific config.

Environment Component	What to Validate	Common Issues
Database	Test data loaded. Correct schema version. Referential integrity intact.	Missing test data. Production data accidentally used.
Integrations	All third-party services accessible (payment, email, SMS).	Integration endpoints pointing to wrong environment.
Test Tools	Test management tool configured. Defect tracker accessible.	JIRA project not set up. Test cases not imported.
Access	All testers have correct access and permissions.	Testers cannot log in or access required features.
Test Data	Test accounts, sample records, and edge-case data ready.	Test data insufficient or stale from a previous sprint.

2.6 Phase 5 — Test Execution

Test Execution is where planned testing work is carried out. Testers execute test cases, compare actual vs expected results, log defects for failures, and track progress against the plan. This phase requires discipline — every result must be recorded, every deviation documented.

Execution Activity	How to Do It	Tool
Execute test case	Follow steps exactly. Compare actual vs expected. Mark Pass/Fail/Blocked.	TestRail, JIRA, Zephyr, Excel
Log defect	Create detailed defect report for every failure. Link to test case.	JIRA, Azure DevOps, Bugzilla
Retest fixed defect	After developer fixes, re-execute the specific test case that failed.	JIRA + Test Management Tool
Regression test	After any code change, execute the regression suite to check for new breakages.	Selenium, Playwright (or manual regression pack)
Report daily status	Update execution counts, defect counts, and blockers at end of each day.	Daily Status Email, Confluence, Slack
Track metrics	Pass rate, defect count by severity, test execution velocity.	Test management dashboard, Excel

2.7 Phase 6 — Test Cycle Closure

Test Cycle Closure is often skipped by junior teams under time pressure, but it is one of the most valuable activities in the STLC. It documents what was done, what was found, what was left out, and what the team should do differently next time.

Closure Activity	Output	Audience
Calculate final metrics	Pass rate, defect removal efficiency, test coverage percentage	QA Lead, Project Manager
Defect analysis	Defects by module, type, root cause. Identify quality hotspots.	Development team, QA Manager
Test Summary Report	Formal document: scope tested, metrics, risks, recommendation	Client, Business Owner, PM, QA Manager
Lessons learned	What worked well, what to improve, action items for next sprint	Entire team
Archive artefacts	Store test cases, reports, evidence in project repository	Future reference, audit

In Practice

In Agile projects the STLC does not happen once per release. It happens in miniature every sprint. A simplified sprint STLC might compress all six phases into two weeks: Requirement analysis on Day 1-2, test case writing on Day 2-5, environment check on Day 3, execution on Day 5-12, and closure on Day 13. Knowing the STLC deeply allows you to scale it up or down to any context.

2.8 Exercises

6. Draw the six-phase STLC diagram. For each phase write the entry criteria, activities, and exit criteria.
7. You have received a user story: 'As a user I want to reset my password via email so I can regain access if I forget it.' Write test conditions for this story using the Requirement Analysis phase techniques.
8. Write a one-page Test Plan for testing the login module of a web application. Include scope, strategy, schedule, resources, and entry/exit criteria.
9. Create a test case in full standard format for the password reset scenario from Exercise 2.
10. Design a test environment checklist for a web application with a database, email integration, and payment gateway.
11. After a sprint, write a Test Cycle Closure report using the following data: 120 test cases planned, 112 executed, 105 passed, 7 failed (3 deferred), 4 blocked. 18 defects logged: 2 P1 (resolved), 8 P2 (6 resolved, 2 deferred), 8 P3 (all resolved).

Quick Quiz

1. List the six phases of the STLC in order and state the primary deliverable of each phase.
2. What is the difference between Test Strategy and Test Plan?
3. A developer marks your defect as Fixed. What is the next STLC activity and how do you perform it?
4. What percentage of total testing effort is typically spent on Test Execution and why?
5. What does Test Cycle Closure achieve that could not simply be skipped under schedule pressure?

Interview Questions

1. Explain the STLC to me as if I have never heard of it. Walk me through each phase.
2. What would you do if a build is delivered to the test environment on Day 8 of a 10-day sprint?
3. How do you know when testing is complete? What signals the end of the Test Execution phase?
4. Describe a Test Plan you have written. What sections did it include and what were the key decisions?
5. What is Defect Removal Efficiency and how do you calculate it?

Chapter Summary

• The STLC is the QA team's own process: six phases from Requirement Analysis to Test Cycle Closure.
• Each phase has defined entry criteria, activities, and exit criteria. Skipping a phase creates debt in later phases.
• The Test Plan is the master QA document. It commits the team to a quality approach and holds everyone accountable.
• Test Execution is not just running tests. It is disciplined tracking, defect logging, retesting, and regression.
• Test Cycle Closure captures lessons learned and metrics that improve every future sprint or release.
• In Agile the STLC compresses into each sprint. The phases still exist — they just happen faster.

Chapter 3 - Defect Life Cycle: Bug Management Mastery

↑ Back to Index

A defect is any behaviour of the software that deviates from the expected result as defined by the requirements, design, or user expectations. Finding a defect is only half the work. The other half is tracking it through its life cycle — from the moment it is discovered to the moment it is resolved and verified. This process is the Defect Life Cycle.

Poor defect management wastes enormous amounts of time. Vague defect descriptions cause developers to reject or misunderstand them. Missing information delays fixes. Untracked defects slip through to production. Mastering the defect life cycle makes you a professional that developers, project managers, and clients trust.

3.1 Defect States and Transitions

State	Who Sets It	Meaning	Next States
New	Tester	Defect has been logged and awaits review	Assigned, Rejected, Deferred
Assigned	QA Lead or Dev Lead	Defect has been assigned to a developer for fixing	Open, Rejected, Deferred
Open	Developer	Developer has started working on the fix	Fixed, Deferred, Not a Bug, Duplicate
Fixed	Developer	Developer believes the fix is complete	Retest
Retest	Tester	Tester is verifying the fix	Closed, Reopened
Closed	Tester	Fix is verified. Defect is resolved.	Terminal — cannot re-open
Reopened	Tester	Fix verification failed. Bug still present.	Open (re-assigned)
Deferred	QA Lead or PM	Fix postponed to future sprint or release	Open (future), Closed (by waiver)
Rejected	Developer or Dev Lead	Developer does not accept defect as valid	Closed (after discussion), Reopened
Not a Bug	Developer or Dev Lead	Behaviour is per design or specification	Closed (after discussion)
Duplicate	QA Lead or Dev Lead	Another defect report for the same issue already exists	Closed (linked to original)

How It Works Internally

Defect tracking tools like JIRA, Azure DevOps, and Bugzilla automate the defect life cycle through workflow configuration. Each state transition triggers notifications: when a developer fixes a defect, the tester is notified automatically for retesting. QA Leads can view all defects in Retest state in a single filter. Learning to configure and query these workflows is a valuable practical skill.

3.2 Severity vs Priority — The Critical Distinction

Severity and Priority are the two most commonly confused concepts in defect management. They are independent dimensions of a defect and must be set separately.

Dimension	Definition	Set By	Scale
Severity	How badly the defect impacts the software's functionality or data	Tester (technical judgement)	Critical/Blocker, Major/High, Minor/Medium, Trivial/Low
Priority	How urgently the defect needs to be fixed relative to business needs	Product Owner or QA Lead (business judgement)	P1 (Immediate), P2 (High), P3 (Medium), P4 (Low)

Scenario	Severity	Priority	Explanation
Login button doesn't work in any browser	Critical	P1	Blocks all users. Fix immediately.
Company logo is broken on the About Us page	Minor	P2	Low technical impact but high client visibility.
Decimal rounding error in an internal report	Major	P3	Affects data accuracy but not used by external users.
Footer copyright year says 2023	Trivial	P1	Visible to all users, regulatory concern in some markets.
Admin-only API throws 500 under extreme load	Major	P4	Affects admin only, load scenario is rare.

3.3 Defect Triage

Defect triage is a structured meeting where the QA team, development team, and project manager review open defects together to make prioritisation decisions. The goal is to ensure every open defect has an agreed action: fix now, fix in next sprint, defer, reject, or close.

Triage Role	Responsibility
QA Lead (Facilitator)	Chairs the meeting. Presents each defect. Ensures all defects are actioned.
QA Tester	Provides technical details, reproduction steps, and additional context.
Developer	Assesses fix complexity, estimates time, proposes solutions.
Project Manager	Makes priority decisions based on business impact and timeline.
Product Owner / BA	Clarifies requirements. Decides whether behaviour is intended.

```
Triage Meeting Agenda (30 minutes, daily):

1. Review previous day's Retest defects — closed or reopened? (5 min)
2. Review New defects logged since last triage (10 min)
   - QA presents each defect
   - Team agrees: Fix / Defer / Reject / Duplicate
   - Developer assigned and priority set
3. Review blocked defects — environment or dependency issues (5 min)
4. Review deferred defects — any change in priority? (5 min)
5. Metrics update: Open defects by severity (5 min)

Output: Updated JIRA with all decisions documented as comments
```

3.4 Defect Metrics

Metric	Formula	Interpretation
Defect Detection Rate (DDR)	Defects found in testing / (Testing + Production) x 100	Target: >95%. Low DDR = testing missed defects.
Defect Rejection Rate	Rejected defects / Total defects logged x 100	Target: <10%. High rate = poor defect writing quality.
Defect Reopen Rate	Reopened defects / Total fixed defects x 100	Target: <5%. High rate = poor fix quality or retesting.
Defect Age (Average)	Sum of (Close Date - Open Date) / Total closed defects	Measures how long defects remain open. Long age = bottleneck.
Defects by Module	Count of defects per module or feature	Identifies quality hotspots requiring extra attention.
Defect Leakage	Defects found in UAT or Production / Total defects x 100	Target: <5% in UAT, <1% in Production.

In Practice

In mature QA organisations a Defect Age report is reviewed weekly. If defects are sitting in Open state for more than 3 days without progress, the QA Lead escalates to the project manager. Defects that sit in Retest state for more than 2 days are followed up directly with the developer. Speed of defect resolution is often the single biggest factor in whether a release makes its date.

3.5 Exercises

12. Draw the complete Defect Life Cycle with all states and transitions. Include the role responsible for each transition.
13. For each of the following scenarios, assign Severity and Priority and justify your choice: (a) The Pay Now button on checkout crashes the app. (b) Hover text on an icon is misspelled. (c) The export to PDF feature is missing in an admin report used by 3 people monthly. (d) The password field accepts passwords of only 1 character due to a validation bug.
14. Write a JIRA defect report for the following: You are testing an e-commerce site. When you add 99 items to the cart and proceed to checkout, the total price shows as negative. The issue happens in Chrome 124 on Windows 11 on the Staging environment on Build 3.4.1.
15. Conduct a mock triage meeting using 5 sample defects provided in the exercises folder. Practice assigning severity, priority, and owner for each.
16. Calculate the following metrics from the given dataset: 80 defects logged, 72 fixed, 8 rejected, 12 found in UAT, 3 found in production after release.

Quick Quiz

1. What is the difference between Severity and Priority? Give an example where a defect has High Priority but Low Severity.
2. Walk me through the Defect Life Cycle from the moment you find a bug to the moment it is closed.
3. What happens when a developer marks your defect as Not a Bug? What do you do next?
4. What is Defect Leakage and why is it the most important defect metric for business stakeholders?
5. What is the purpose of a defect triage meeting and who should attend?

Interview Questions

1. Describe the defect life cycle used in your last project. Were there any states unique to that project?
2. A developer rejects 30% of your defects as Not a Bug. How do you handle this situation?
3. How do you prioritise which defects to retest first when you have 20 defects in Retest state and the release is tomorrow?
4. What is the Defect Removal Efficiency metric and how do you improve it?
5. Tell me about the most impactful defect you have found in your career. How did you discover it and what was the business impact?

Chapter Summary

• The Defect Life Cycle tracks every bug from New to Closed. Every state transition has a responsible party.

• Severity measures technical impact. Priority measures business urgency. They are independent and must both be set.
• Defect triage is a daily discipline that prevents defects from stagnating and ensures release decisions are informed.
• Defect metrics tell the story of quality. DDR, rejection rate, age, and leakage are the four most important metrics.
• A rejected defect is not a failure — it is a conversation. Resolve disputes with evidence from requirements, not opinion.
• Mastering defect management builds your credibility with development teams and project managers alike.

Chapter 4 - Types of Testing: Choosing the Right Approach

↑ Back to Index

One of the most common questions in QA interviews is: what types of testing do you know? But the more important question is: how do you decide which type to use and when? Knowing the names of testing types is basic knowledge. Understanding when, why, and how to apply each type is what makes you a skilled tester.

Testing types can be categorised along several dimensions: by functional vs non-functional, by level of the stack (unit vs integration vs system vs acceptance), by degree of scripting (scripted vs exploratory), and by purpose (regression vs smoke vs security vs performance). This chapter gives you a complete mental model for all these dimensions.

4.1 Functional vs Non-Functional Testing

Category	Tests Whether	Examples	Tools
Functional Testing	The software does what it is supposed to do according to requirements	Login works, order is placed, email is sent, calculation is correct	Manual, Selenium, Playwright, Cucumber
Non-Functional Testing	How well the software performs, scales, secures, and operates	Performance, security, accessibility, usability, compatibility, reliability	JMeter, k6, OWASP ZAP, Axe, BrowserStack

4.2 Functional Testing Types

Testing Type	Purpose	When to Use	Level
Unit Testing	Verify smallest testable unit of code (function/method) in isolation	During development, before integration	Component
Integration Testing	Verify that two or more components work correctly together	After unit testing, when modules are connected	Integration
System Testing	Verify the fully integrated system against requirements	After integration testing, on complete build	System
Acceptance Testing (UAT)	Verify the system meets business user needs and expectations	Before release, by business users	Acceptance

Testing Type	Purpose	When to Use	Level
Smoke Testing	Quick check that the most critical functionality works after a build	After every new build delivery	System
Sanity Testing	Narrow, focused check that a specific bug fix or change works	After a targeted fix is deployed	System
Regression Testing	Ensure existing functionality is not broken by new changes	After any code change	System/Integration
End-to-End Testing	Verify complete business workflows from start to finish	Pre-release, for critical user journeys	System/Acceptance

4.3 Smoke vs Sanity vs Regression — The Critical Differences

These three testing types are the most commonly confused at the junior level. Here is a definitive comparison:

Aspect	Smoke Testing	Sanity Testing	Regression Testing
Purpose	Verify build is stable enough for testing	Verify a specific fix or change works	Verify nothing new is broken
Scope	Broad and shallow — covers critical paths	Narrow and deep — covers one specific area	Broad and thorough — covers all previous functionality
Who runs it	QA Engineer (or automated pipeline)	QA Engineer who logged the defect	QA team or automated suite
When it runs	After every new build delivery	After a specific bug fix is deployed	After any code change, before release
If it fails	Reject the build, request a new one	Reopen the defect, notify developer	Log new regression defects, assess release risk
Duration	15-30 minutes (or automated)	5-15 minutes (focused area)	Hours to days (full regression suite)

4.4 Exploratory and Ad-Hoc Testing

Exploratory testing is simultaneous learning, test design, and execution. The tester uses their knowledge of the system and intuition to investigate areas most likely to contain defects, without pre-written scripts. It is not unstructured — it is guided by a charter.

```
Exploratory Testing Charter Example:

Session Title  : Explore the Checkout and Payment Flow
Duration       : 90 minutes (time-boxed)
Focus Area     : Cart > Checkout > Payment > Confirmation
Test Mission   : Find unexpected errors, edge cases, and usability issues
                 in the end-to-end purchase flow

Notes to guide exploration:
- Try unusual quantities (0, 999, decimals)
- Test with different payment methods in sequence
- Interrupt flow mid-checkout (browser back, refresh, close)
- Test with and without promo codes
- Try concurrent sessions in same account

Debrief Output: 3 defects logged (1 P2, 2 P3), 2 questions raised to BA
Session Notes : [Written during session]
```

4.5 Non-Functional Testing Overview

NFT Type	What It Tests	Key Metrics	Common Tool
Performance Testing	Response time, throughput, resource usage under load	Response time < 2s, throughput > 500 TPS	JMeter, k6, Gatling
Load Testing	System behaviour under expected peak load	All users served within SLA at peak load	JMeter, Locust
Stress Testing	System behaviour beyond normal capacity	Find breaking point and recovery behaviour	k6, Gatling
Security Testing	Vulnerability identification, authentication, authorisation flaws	0 critical OWASP Top 10 vulnerabilities	OWASP ZAP, Burp Suite
Usability Testing	How easy and intuitive the system is to use	Task completion rate, error rate, user satisfaction	UserTesting, Lookback
Accessibility Testing	Compliance with WCAG 2.1 standards for disabled users	0 critical accessibility violations	Axe, WAVE, Lighthouse
Compatibility Testing	Correct function across browsers, OS, devices	Works in agreed browser/device matrix	BrowserStack, Sauce Labs
Reliability Testing	System stability over extended periods (soak testing)	0 memory leaks, consistent performance over 24+ hours	JMeter, AWS Load Testing

In Practice

In most Agile projects you will perform functional, regression, and smoke testing every sprint. Performance and security testing typically happen at dedicated milestones or pre-release cycles. UAT happens at the end of a development phase or before a major release. Exploratory testing should be timebox-allocated in every sprint — typically 2-4 hours per week — to catch what scripted tests miss.

4.6 How to Choose the Right Testing Type

- After a new build: always smoke test first. If smoke fails, reject the build and do not waste time on deeper testing.
- After a bug fix: sanity test the fix, then run the relevant regression subset to check for unintended side effects.
- Before a major release: run the full regression suite plus UAT plus exploratory sessions on high-risk areas.
- When requirements are new or unclear: use exploratory testing to discover behaviour before writing formal test cases.
- When the system is stable and well-understood: scripted regression testing is most efficient.
- When testing a critical user journey: end-to-end testing ensures the complete workflow works, not just individual components.

4.7 Exercises

17. Create a decision table mapping 5 project scenarios to the most appropriate testing type and explain your choice.
18. Write a 90-minute exploratory testing charter for the user profile settings section of a social media application.
19. A developer tells you a bug has been fixed. Describe the exact sequence of testing activities you would perform: smoke test, sanity test, or full regression? Why?
20. Research the OWASP Top 10 vulnerabilities list. For each vulnerability, describe one test you could perform manually to check for it.
21. Compare Selenium and Playwright for automating regression tests. Create a 1-page comparison covering: setup, supported languages, speed, community, and best use case.

Quick Quiz

1. What is the difference between Smoke Testing and Regression Testing? When would you run each?
2. Is Unit Testing the responsibility of the QA team? Explain your answer.
3. Describe Exploratory Testing. How is it different from Ad-hoc Testing?
4. A new build is delivered at 4pm and the release is at 6pm. What testing type do you perform and what does pass/fail mean?
5. Name four types of non-functional testing and state what metric each one measures.

Interview Questions

1. What types of testing have you performed in your career? Walk me through your experience with each.
2. Explain the difference between Smoke, Sanity, and Regression testing to a non-technical project manager.
3. When would you choose exploratory testing over scripted testing? Give a specific example.
4. What non-functional testing have you done? How did you set up and execute performance tests?
5. A sprint delivers 5 user stories and fixes 8 defects. What types of testing would you plan and in what order?

Chapter Summary

• Testing types span functional and non-functional dimensions. Both are essential for production-quality software.

• Smoke tests the build stability. Sanity tests the specific fix. Regression tests nothing new is broken. These are distinct and sequential.

• Exploratory testing uses tester intelligence to find bugs that scripts miss. It should be part of every sprint.

• Non-functional testing (performance, security, accessibility) is not optional — it is a key component of release quality.

• Choosing the right testing type for the right context is a core QA skill that differentiates good testers from great ones.

• Always smoke test first after a new build. Never waste time on deeper testing if the build cannot even launch.

Chapter 5 - Test Case Design Techniques

↑ Back to Index

Test case design is the intellectual core of manual testing. The question is never 'should we test this?' but 'how do we test this efficiently and completely?' Without systematic design techniques, testers default to intuition — which misses entire categories of defects and duplicates effort on cases that test the same thing.

The techniques in this chapter come from decades of research into where software fails most often. They are not theoretical — every professional test suite should use multiple techniques in combination. Mastering them separates junior testers who write 200 test cases for a login page from senior testers who write 30 cases that achieve greater coverage.

5.1 Boundary Value Analysis (BVA)

Software most often fails at boundaries — the edges of valid and invalid input ranges. BVA directs test effort to exactly these boundary points rather than testing arbitrary values in the middle of a range.

BVA Rule	Points to Test	Example: Age field (18-60)
Just below lower boundary	Min - 1	17 (invalid — expect error)
At lower boundary	Min	18 (valid — expect acceptance)
Just above lower boundary	Min + 1	19 (valid — expect acceptance)
Just below upper boundary	Max - 1	59 (valid — expect acceptance)
At upper boundary	Max	60 (valid — expect acceptance)
Just above upper boundary	Max + 1	61 (invalid — expect error)

```
BVA Applied — Password Length (8-16 characters):

TC-BVA-001: Enter password of 7 chars  => Expected: Validation error
TC-BVA-002: Enter password of 8 chars  => Expected: Accepted
TC-BVA-003: Enter password of 9 chars  => Expected: Accepted
TC-BVA-004: Enter password of 15 chars => Expected: Accepted
TC-BVA-005: Enter password of 16 chars => Expected: Accepted
TC-BVA-006: Enter password of 17 chars => Expected: Validation error

Total: 6 test cases cover the boundary risk completely.
Without BVA a tester might only test 10 chars and miss boundary bugs.
```

5.2 Equivalence Partitioning (EP)

EP divides the input space into groups (partitions) where all values in a group are expected to behave identically. Instead of testing every possible value, you test one representative value from each partition. This dramatically reduces the number of test cases while maintaining coverage.

Partition Type	Description	Example: Email field
Valid partition	Values the system should accept	user@domain.com, user.name@company.co.uk
Invalid partition 1	Missing @ symbol	userdomain.com, user@, @domain.com
Invalid partition 2	Missing domain	user@, user@.com
Invalid partition 3	Empty input	'' (blank field)
Invalid partition 4	Special characters not allowed	user!#$@domain.com

By combining BVA and EP you get maximum coverage with minimum test cases. BVA tells you to test at the edges of each partition. EP tells you one test case per partition is sufficient for values in the middle.

5.3 Decision Table Testing

Decision tables are used when multiple conditions combine to produce different outcomes. They force you to consider every combination of inputs and ensure each combination has a defined expected result. They are essential for business rule testing.

```
Decision Table Example — Loan Eligibility:

Conditions:
  C1: Credit Score >= 700   (Y/N)
  C2: Annual Income >= 5L   (Y/N)
  C3: Existing Loans < 2    (Y/N)

Rules:    R1   R2   R3   R4   R5   R6   R7   R8
C1:        Y    Y    Y    Y    N    N    N    N
C2:        Y    Y    N    N    Y    Y    N    N
C3:        Y    N    Y    N    Y    N    Y    N
           --   --   --   --   --   --   --   --
Action:   APP  REV  REV  REJ  REV  REJ  REJ  REJ

APP = Approved, REV = Manual Review, REJ = Rejected

Each column = one test case. 8 test cases cover all combinations.
```

5.4 State Transition Testing

State transition testing is used when a system moves through different states based on events or inputs. It ensures all valid transitions are tested and all invalid transitions are properly rejected.

Current State	Event / Input	Next State	Output / Action
Idle	User inserts card	Card Inserted	Welcome screen shown
Card Inserted	User enters correct PIN	Authenticated	Main menu shown
Card Inserted	User enters wrong PIN	PIN Failed (1)	Re-enter PIN prompt
PIN Failed (1)	User enters wrong PIN again	PIN Failed (2)	Re-enter PIN prompt
PIN Failed (2)	User enters wrong PIN again	Card Blocked	Card retained, message shown
Authenticated	User selects Withdraw	Transaction In Progress	Amount entry screen
Transaction In Progress	User enters amount and confirms	Dispensing Cash	Cash dispensed
Dispensing Cash	Cash collected	Idle	Goodbye message shown

```
State Transition Test Cases (ATM example):

TC-ST-001: Idle -> Insert Card -> Card Inserted [VALID]
TC-ST-002: Card Inserted -> Correct PIN -> Authenticated [VALID]
TC-ST-003: Card Inserted -> Wrong PIN x3 -> Card Blocked [VALID]
TC-ST-004: Authenticated -> Withdraw -> Dispensing Cash [VALID]
TC-ST-005: Idle -> Enter PIN (no card) -> Error: No card [INVALID TRANSITION]
TC-ST-006: Card Blocked -> Enter PIN -> Error: Card blocked [INVALID]
```

5.5 Pairwise (All-Pairs) Testing

When a system has multiple input parameters each with multiple values, the number of combinations explodes quickly. Pairwise testing ensures every pair of parameter values is tested at least once, catching the vast majority of bugs with a fraction of the test cases.

```
Pairwise Example — Web App Compatibility:

Parameters:
  OS:      Windows, macOS, Linux          (3 values)
  Browser: Chrome, Firefox, Safari        (3 values)
  Screen:  1080p, 4K, Mobile             (3 values)

Full combinations: 3 x 3 x 3 = 27 test cases

Pairwise (using PICT tool or AllPairs algorithm):

TC  OS        Browser   Screen
1   Windows   Chrome    1080p
2   Windows   Firefox   4K
```

```
3   Windows   Safari    Mobile
4   macOS     Chrome    4K
5   macOS     Firefox   Mobile
6   macOS     Safari    1080p
7   Linux     Chrome    Mobile
8   Linux     Firefox   1080p
9   Linux     Safari    4K

9 test cases cover all pairs. Saves 66% of test effort.
```

5.6 Error Guessing

Error guessing is an experience-based technique where the tester uses knowledge of past defects, common programming errors, and application behaviour to identify likely defect areas. It is not random — it is informed intuition built through practice.

- Empty inputs: Most applications fail when fields are left blank. Always test empty strings, null values, and zero.
- Special characters: SQL injection, XSS, and encoding bugs often appear when special characters are entered in input fields.
- Maximum input length: What happens when you enter 1000 characters in a field expecting 50? Buffer overflows are common.
- Negative numbers: Financial and quantity fields often fail with negative values.
- Duplicate submission: Double-clicking a submit button can create duplicate records or process payments twice.
- Concurrent operations: Two users editing the same record simultaneously can cause data corruption.
- Expired sessions: What happens when a user tries to submit a form after their session has timed out?
- Interrupted flows: What happens if you close the browser mid-checkout, mid-payment, or mid-upload?

5.7 Choosing the Right Technique

Scenario	Recommended Technique(s)
Numeric input with min/max range	BVA + EP
Text field with valid/invalid categories	EP + Error Guessing
Business rules with multiple conditions	Decision Table Testing
System with modes, states, or workflows	State Transition Testing
Multi-parameter compatibility testing	Pairwise Testing
Brand new feature with no history	Exploratory + Error Guessing
Complex user registration form	BVA + EP + Error Guessing + Decision Table

5.8 Exercises

22. Apply BVA to a date field that accepts dates between 01/01/1900 and 31/12/2099. List all boundary test cases.
23. Create equivalence partitions for a mobile phone number field in an Indian application. Include all valid and invalid partitions with examples.
24. Build a decision table for a hotel booking discount: Loyalty members get 10% off. Bookings over 7 nights get 15% off. Members booking over 7 nights get 20% off. Non-members booking 7 nights or less get no discount.
25. Draw a state transition diagram for an online order (Placed > Confirmed > Packed > Shipped > Delivered > Cancelled). Write 6 test cases including both valid transitions and invalid transition attempts.
26. Use pairwise technique to reduce a test matrix for a report generation feature: Format (PDF/Excel/CSV), Date Range (Daily/Weekly/Monthly), User Type (Admin/Manager/User).

Quick Quiz

1. What is Boundary Value Analysis? Why do bugs most often occur at boundaries?
2. Explain Equivalence Partitioning with an example. How many test cases do you need per partition?
3. You are testing an insurance premium calculator with 4 binary conditions. How many rules does your decision table have?
4. Describe a scenario where State Transition Testing is the most appropriate technique to use.
5. What is Pairwise Testing and what problem does it solve?
6. What is Error Guessing and how is it different from Exploratory Testing?

Interview Questions

1. Walk me through how you would design test cases for a password change feature using at least 3 design techniques.
2. Explain Boundary Value Analysis to me as if I am a developer who has never heard of it.
3. How do you ensure your test suite achieves adequate coverage without writing thousands of redundant test cases?
4. Give me an example of a decision table you have used in a real project.
5. What is your approach to test case design when requirements are ambiguous or incomplete?

Chapter Summary

• BVA directs testing effort at boundary values where software most commonly fails.
• EP divides inputs into partitions — one test case per partition is sufficient coverage.
• Decision tables capture all combinations of business rules systematically and completely.
• State transition testing verifies all valid state changes and rejects all invalid ones.
• Pairwise testing reduces combinatorial explosion while maintaining high defect detection.
• No single technique covers everything. Combine 2-3 techniques for any given feature to achieve thorough coverage.

Chapter 6 - Requirement Analysis and RTM

↑ Back to Index

Requirements are the foundation on which testing is built. If requirements are wrong, ambiguous, or untestable, no amount of testing skill can save the project. The ability to critically analyse requirements, identify gaps, and transform them into testable conditions is one of the highest-value skills a QA professional can develop.

The Requirement Traceability Matrix is the living document that connects requirements to test cases, test cases to execution results, and results to business outcomes. It is the single artefact that answers the question: have we tested everything the business asked for?

6.1 Characteristics of Good Requirements

Characteristic	Definition	Good Example	Poor Example
Complete	Covers all scenarios without gaps	User receives email within 5 minutes of registration	User receives email after registration
Clear	Unambiguous, only one interpretation possible	Password must be 8-16 characters	Password must be strong
Testable	Can be verified with a specific test	Page loads within 3 seconds on 10Mbps connection	Page should load fast
Consistent	Does not contradict other requirements	Session expires after 30 minutes (aligned with security policy)	Session never expires (contradicts security policy)
Feasible	Technically and commercially achievable	System must support 1000 concurrent users	System must support infinite users
Traceable	Can be linked to a business objective	REQ-015 implements FR-003 from Business Requirements Document v2	Added by dev team, no business requirement reference

6.2 Requirement Review Techniques

Reviewing requirements is a formal quality activity. It should be structured, documented, and produce actionable outputs — not just a casual read-through.

Review Type	Description	Best For
Walkthrough	Author walks the team through the document. Team raises questions and suggestions.	Early drafts, collaborative refinement

Review Type	Description	Best For
Inspection (Fagan)	Formal, structured review with defined roles (moderator, author, reviewer, recorder). Uses checklist.	High-risk or complex requirements
Peer Review	One or two peers read and comment independently	Standard requirements in Agile projects
Three Amigos	BA, Developer, and QA review each user story together. Focus on shared understanding.	Agile user stories before sprint planning

6.3 Testability Analysis

Not all requirements are testable as written. A testability analysis identifies which requirements need to be rewritten or clarified before test cases can be written. Common testability issues include:

- Subjective terms: Words like fast, easy, user-friendly, and reliable are untestable without a measurable definition. Ask for specific metrics.
- Missing acceptance criteria: A requirement without acceptance criteria cannot be tested. Every requirement must specify what pass looks like.
- Compound requirements: A requirement that covers multiple behaviours should be split. It is impossible to trace a compound requirement cleanly to test cases.
- Negative requirements: Requirements stating what the system should NOT do are hard to test completely. Request positive equivalents where possible.
- Undefined conditions: Requirements that use terms like sometimes, usually, or may are ambiguous. Every condition must be defined explicitly.

6.4 RTM Structure and Purpose

The Requirement Traceability Matrix maps each requirement to one or more test cases. It serves three purposes: coverage verification (are all requirements tested?), impact analysis (if a requirement changes, which test cases are affected?), and release evidence (what was tested and what was the result?).

RTM Column	Content	Example
Requirement ID	Unique identifier from requirements document	REQ-015
Requirement Description	Brief description of the requirement	User can reset password via email link
Priority	Business priority of the requirement	High
Test Case ID(s)	All test cases that verify this requirement	TC-045, TC-046, TC-047, TC-048

RTM Column	Content	Example
Test Case Description	Brief description of what each TC covers	Valid email, invalid email, expired link, already used link
Execution Status	Current status of each test case	Pass, Pass, Fail, Not Run
Defect ID(s)	Any defects linked to failures	DEF-112
Remarks	Notes or exceptions	TC-048 blocked by DEF-089, will run after fix

```
Sample RTM (4 requirements, partial):

Req ID  | Description              | Priority | TC IDs          | Status          |
Defects
--------|--------------------------|----------|-----------------|-----------------|-
-------
REQ-001 | User login with email    | High     | TC-001, TC-002  | Pass, Pass      |
-
REQ-002 | Password reset via email | High     | TC-003 to TC-007| Pass,Pass,Fail,-|
DEF-012
REQ-003 | Profile photo upload     | Medium   | TC-008, TC-009  | Pass, Blocked   |
DEF-015
REQ-004 | Account deletion         | Low      | TC-010          | Not Run         |
-

Coverage: 4/4 requirements have test cases (100% coverage)
Execution: 7/10 test cases run. 5 Pass, 1 Fail, 1 Blocked, 3 Not Run.
Gaps: REQ-004 not yet executed. Requires attention before release.
```

6.5 Maintaining the RTM

- Update the RTM every time a requirement changes. A stale RTM is worse than no RTM — it gives false confidence.
- Link every test case back to a requirement. Orphan test cases (no requirement link) should be questioned or removed.
- Review RTM coverage before every sprint ends. Identify untested or partially tested requirements.
- Use the RTM for impact analysis when changes are requested. It shows exactly which test cases need to be updated or re-run.
- At release, the RTM provides formal evidence of test coverage to clients, auditors, or regulatory bodies.

6.6 Exercises

27. Take the following requirement and identify 5 testability issues: 'The application should respond quickly and handle a large number of users efficiently across all major browsers.'

28. Rewrite the requirement from Exercise 1 as 3 separate, testable, measurable requirements.
29. Create a full RTM for the following 5 requirements: (a) User can register with email and password. (b) User receives a confirmation email within 5 minutes. (c) User can log in with correct credentials. (d) User is shown an error after 3 failed login attempts. (e) User session expires after 30 minutes of inactivity.
30. A new requirement is added after test cases are written: Password must contain at least one uppercase letter and one number. Update the RTM and identify which existing test cases need to be changed.
31. Conduct a Three Amigos review for the user story: 'As a customer, I want to save my cart so I can come back to it later.' Write 5 questions the QA engineer should raise and propose acceptance criteria for 3 key scenarios.

Quick Quiz

1. What are the six characteristics of a good requirement? Give an example of a requirement that violates each.
2. What is the difference between a Review and an Inspection in the context of requirements review?
3. What is an orphan test case and why is it a problem?
4. How do you use the RTM to perform impact analysis when a requirement changes?
5. A stakeholder says 100% requirement coverage is achieved in the RTM. What does this actually mean and what could it NOT tell you?

Interview Questions

1. How do you review requirements? Walk me through your approach when you receive a new user story.
2. Give me an example of a requirement you have received that was untestable. How did you handle it?
3. What is the RTM and how have you used it in a real project?
4. How do you ensure complete test coverage of all requirements in a sprint with changing scope?
5. Describe a situation where a missing or ambiguous requirement caused a defect in production. What did you do?

Chapter Summary

• Requirements are the foundation of testing. Poor requirements lead to poor coverage no matter how well you test.
• Good requirements are complete, clear, testable, consistent, feasible, and traceable.
• Testability analysis identifies requirements that must be refined before test cases can be written.
• The RTM is the living link between requirements and test cases. It proves coverage and enables impact analysis.
• Maintain the RTM throughout the project. A stale RTM gives false confidence and fails audits.
• The Three Amigos technique — BA, Dev, QA — is the most effective way to catch requirement gaps before development starts.

Chapter 7 - Defect Reporting and Defect Triage

↑ Back to Index

A defect report is a professional document. It must communicate precisely what went wrong, how to reproduce it, what was expected, what actually happened, and how impactful the issue is — clearly enough that a developer who was not present when the bug was found can understand and fix it without asking you a single question.

This chapter covers the art and science of writing defect reports that get actioned quickly, conducting triage meetings efficiently, and using defect data to drive quality improvement. These skills directly impact how developers, project managers, and clients perceive the quality of the QA team.

7.1 Anatomy of a Professional Defect Report

Field	What to Write	Common Mistake
Title / Summary	Concise, specific, searchable. State the symptom and affected area.	Writing 'Login is broken' instead of 'Login button throws HTTP 500 when password contains special characters'
Environment	Browser version, OS, application version, build number, server/environment name.	Writing 'Chrome' instead of 'Chrome 124.0.6367.82 on Windows 11 Pro 22H2 on QA environment Build 2.4.1'
Severity	Technical impact: Critical / Major / Minor / Trivial	Setting all defects to Critical to get attention
Priority	Business urgency: P1 / P2 / P3 / P4	Confusing with Severity
Steps to Reproduce	Numbered, precise, exact steps. Anyone should be able to reproduce the bug.	Writing 'Go to the page and click something' — too vague to reproduce
Expected Result	What the system SHOULD do based on requirements or common sense.	Describing what the system does (actual result) as the expected result
Actual Result	What the system ACTUALLY does. Be specific.	Writing 'It doesn't work' — meaningless without detail
Attachments	Screenshots, screen recordings, log files, network traces.	Logging defects without any visual evidence
Test Case ID	Link to the test case that found this defect.	Not linking, making the defect untraceable to coverage
Additional Info	Frequency (always/intermittent), workaround if known, similar defects.	Not noting intermittent behaviour — wastes developer time

7.2 Worked Example: Well-Written vs Poorly-Written Defect

```
POORLY WRITTEN DEFECT:
Title: Cart is broken
Steps: 1. Go to cart  2. Something goes wrong
Expected: It should work
Actual: It doesn't work
Severity: Critical
Attachments: None

Result: Developer cannot reproduce. Defect rejected. Time wasted.

---

PROFESSIONALLY WRITTEN DEFECT:
Title: Cart total shows negative value when quantity exceeds 99 items

Environment:
  Browser: Chrome 124.0 on Windows 11
  Environment: QA (qa.company.com)
  Build: v3.4.1 (deployed 14-Jun-2026 09:00)

Severity: Major | Priority: P2

Steps to Reproduce:
  1. Log in as user: testuser@example.com / Pass@123
  2. Navigate to Product Listing page
  3. Search for 'Blue Widget' (SKU: BW-100)
  4. Add product to cart
  5. On Cart page, change quantity to 100
  6. Click 'Update Cart' button
  7. Observe the Total Price field

Expected Result:
  Total Price = 100 x Rs.499 = Rs.49,900

Actual Result:
  Total Price = -Rs.49,401 (negative value displayed)

Frequency: Reproducible 100% of the time with quantity >= 100

Attachments: cart_bug_screenshot.png, cart_api_response.json
Test Case: TC-CART-047

Result: Developer reproduces in 5 minutes. Fixed same day.
```

7.3 Defect Triage Process

Defect triage is a structured collaborative meeting that ensures every open defect is reviewed, prioritised, assigned, and actioned. Without triage, defects pile up, developers work on wrong priorities, and project managers lack visibility.

Triage Phase	Activities	Duration
Preparation (QA before meeting)	Filter all New and Reopened defects. Sort by severity. Prepare business context for each.	10-15 min
New Defects Review	QA presents each defect. Team agrees on severity, priority, assignment, and target sprint.	15-20 min
Deferred Defect Review	Review deferred defects from previous sprint. Any change in priority?	5 min
Metrics Update	Review open defect count by severity. Identify trends and risks.	5 min
Actions & Documentation	QA Lead updates all JIRA tickets with triage decisions and notes.	5 min post-meeting

7.4 Handling Rejected Defects

A developer or dev lead rejects a defect as Not a Bug or Invalid. This happens in every project. The professional response is not to escalate immediately or accept silently — it is to engage in evidence-based discussion.

32. Understand the rejection reason: Ask the developer to explain why the behaviour is expected. Document their response.
33. Check the requirements: Look at the relevant requirement, acceptance criteria, or design document. Does the behaviour match?
34. Check the design mockup: Compare actual behaviour to the approved UI design or wireframe.
35. If the requirements are silent: Escalate to the Business Analyst or Product Owner to clarify the expected behaviour.
36. If you still disagree: Escalate to the QA Lead and PM with evidence. Never close a disputed defect without written consensus.
37. If the developer is correct: Accept the rejection graciously, update the defect status to Closed (Not a Bug), and note the clarification for future test cases.
38. If your requirements are confirmed: Reopen the defect with the requirements document reference quoted.

7.5 Defect Reporting Tools

Tool	Best For	Key Features
JIRA	Enterprise teams, Agile projects	Custom workflows, rich filters, dashboards, integrations, automation
Azure DevOps (ADO)	Microsoft stack teams, Agile + DevOps	Integrated with CI/CD pipelines, test plans, git repos
Bugzilla	Open-source teams, smaller projects	Free, flexible, basic workflow
GitHub Issues	Open-source and developer-led teams	Lightweight, integrated with git, labels and milestones
TestRail	Test management + defect linking	Full STLC management, links test cases to JIRA defects
Mantis BT	Small teams, simple projects	Free, email integration, basic customisation

7.6 Exercises

39. Find 3 real defects in any publicly available application (mobile app, website). Write a professional defect report for each in the standard format.
40. Take the following vague defect description and rewrite it professionally: 'The search feature sometimes doesn't show results when you type something in the search box on the homepage.'
41. Roleplay a triage meeting with a partner. Use 5 sample defects from the exercises pack. Assign severity, priority, and owner for each. Document the triage decisions.
42. A developer rejects your defect: 'The application accepts a 1-character username.' The rejection says: By design. Write your professional response citing the appropriate requirements document (provided in the exercises pack).
43. Create a defect trend report for a sprint using the following data: Week 1: 12 new, 5 fixed, 2 deferred. Week 2: 8 new, 10 fixed, 3 reopened. Week 3: 4 new, 8 fixed, 0 reopened. Visualise the trend and write a 3-sentence interpretation.

Quick Quiz

1. What are the 10 mandatory fields in a professional defect report?
2. What is the difference between severity and priority and who sets each?
3. A developer marks your defect as Not a Bug. Walk me through your response process step by step.
4. What is the purpose of a defect triage meeting and how often should it be held?
5. What does a Defect Rejection Rate of 25% indicate about the quality of the QA team's defect reports?

Interview Questions

1. Walk me through a defect report you are proud of. What made it effective?
2. How do you handle a situation where a developer consistently rejects your defects?

3. Describe the defect triage process you have participated in. What was your role?

4. How do you ensure defect reports contain enough information for developers to reproduce the issue without follow-up questions?

5. Tell me about a critical defect you found late in a project. How did you communicate it and what was the outcome?

Chapter Summary

• A defect report is a professional document. Every field has a purpose. Incomplete reports slow down fixes.

• The title alone must tell a developer where the bug is, what went wrong, and under what condition.

• Severity and Priority are distinct. Set both correctly. Setting everything to Critical destroys credibility.

• Defect triage is a daily discipline. Without it defects accumulate, priorities shift, and release decisions are made blindly.

• When a defect is rejected, respond with evidence — requirements, designs, specifications — not opinion.

• The quality of your defect reports is a direct reflection of your professionalism. Make every report count.

Chapter 8 - Test Execution, Metrics and Reporting

↑ Back to Index

Test execution is not just running test cases and marking them pass or fail. It is a managed, disciplined process that generates evidence of quality, tracks progress against plan, escalates risks, and culminates in a Test Summary Report that gives stakeholders the confidence to make release decisions.

Test metrics are the language that QA uses to communicate with people who do not test — project managers, business owners, and clients. Without metrics, quality is invisible. With the right metrics, communicated clearly, QA becomes a strategic function that drives release confidence.

8.1 Test Execution Best Practices

- Execute in priority order: Start with P1 and smoke test cases. If they fail, stop and raise an immediate blocker.
- Record every result: Every test case executed must have a recorded result — Pass, Fail, or Blocked. Never leave results blank.
- Link every failure to a defect: Every failed test case must have a corresponding defect in the defect tracker, linked back to the test case.
- Follow steps exactly: Do not improvise during execution. If you deviate from the steps, the test case must be updated first.
- Test in the agreed environment: Executing on the wrong environment (e.g. developer's machine instead of QA server) produces invalid results.
- Maintain independence: Do not let developers watch you test. It changes your behaviour and introduces observer effect bias.
- Time-box exploratory sessions: For every 4 hours of scripted execution, include 30-60 minutes of exploratory testing to find edge cases.

8.2 Handling Blocked Test Cases

Blocked test cases are those that cannot be executed due to an environmental issue, missing test data, or a blocking defect. Blocked cases must be tracked, escalated, and resolved — not ignored.

Block Type	Example	Resolution Action
Environment issue	Test environment is down or inaccessible	Notify lead immediately. Log environment defect. Resume when resolved.
Missing test data	Required test account does not exist	Request data from dev team. Use data provisioning script.
Blocking defect	A P1 defect prevents access to a feature under test	Log the blocked TC and link to blocking defect. Resume after fix.

Block Type	Example	Resolution Action
Build instability	Intermittent crashes make execution unreliable	Invoke suspension criteria. Request a new stable build.
Missing access	Tester does not have permission to access the feature	Request access from system admin or project manager.

8.3 Key Test Metrics

Metric	Formula	Target	Tells You
Test Execution Rate	Executed / Total Planned x 100	>90% by end of cycle	How much of the planned testing is complete
Test Pass Rate	Passed / Total Executed x 100	>95% for release	Quality of the build being tested
Defect Detection Rate (DDR)	Defects in testing / (Testing + Production) x 100	>95%	How effective testing is at catching bugs pre-release
Blocked Test Rate	Blocked / Total Planned x 100	<5%	Impact of environment or data issues on test progress
Defect Density	Defects found / Story Points or Feature Points	Trending down over sprints	Quality of development output
Test Coverage	Requirements with test cases / Total requirements x 100	100% for P1/P2 reqs	Completeness of test design against requirements
Defect Removal Efficiency (DRE)	Pre-release defects / (Pre + Post-release) x 100	>95%	Overall effectiveness of the QA process
Test Execution Velocity	Test cases executed per day	Trending stable or up	Team productivity and sustainability

8.4 Daily Status Reporting

```
Sample Daily Test Status Update (email or Slack):

SPRINT 14 - QA STATUS UPDATE - Day 8 of 10

EXECUTION SUMMARY:
  Total Test Cases : 120
  Executed Today   : 18   |  Cumulative: 98 (82%)
  Passed           : 15   |  Failed: 2  |  Blocked: 1
```

```
  Cumulative Pass  : 91/98 = 92.9%

DEFECTS:
  New defects today : 2 (1 P2, 1 P3)
  Total Open        : 6 (0 P1, 3 P2, 3 P3)
  Fixed & Retested  : 4 today
  In Retest         : 3

BLOCKERS:
  DEF-089 (P1 - Payment API down) - blocking 3 test cases
  ETA for fix: Tomorrow 10am (developer confirmed)

PLAN FOR TOMORROW:
  Execute remaining 22 test cases
  Retest 3 blocked cases after DEF-089 fix
  Complete regression suite (14 cases)

RISK: If DEF-089 not fixed by 10am, release date at risk.
  Recommend: PM to follow up with dev lead tonight.
```

8.5 Sprint Test Summary Report

The Test Summary Report is the formal closing document of a test cycle. It is submitted to the project manager, business stakeholder, and QA manager at the end of each sprint or release cycle.

Section	Content	Length
Executive Summary	One paragraph: overall quality assessment and release recommendation	3-5 sentences
Test Scope	What was tested and what was explicitly out of scope	Bullet list
Test Execution Summary	Metrics table: planned, executed, passed, failed, blocked	1 table
Defect Summary	Defects by severity, status, module	1-2 tables + chart
Requirements Coverage	RTM coverage: how many requirements were tested and at what pass rate	1 table
Risk and Issues	Open risks, deferred defects, outstanding items	Bullet list
Release Recommendation	Go / No-Go with clear justification based on exit criteria	1 paragraph

Section	Content	Length
Test Artefacts	Links to test cases, defect reports, execution logs in repository	Reference list

8.6 Release Recommendation Framework

Condition	Go	No-Go
P1/Critical defects	0 open	Any open P1 = No-Go
P2/High defects	3 or fewer, all with mitigation plan	More than 3 open P2 without plan = No-Go
Test Execution Rate	Greater than 90% of planned cases executed	Below 80% = No-Go
Pass Rate	Greater than 95%	Below 90% = No-Go unless deferred are low risk
Automation Coverage	Greater than 80% regression covered	Below 60% = risk flag (not always No-Go)
UAT Sign-off	Business owner signed off formally	No sign-off = No-Go
Performance SLA	Response time within agreed thresholds	SLA breached on critical paths = No-Go

8.7 Exercises

44. Execute a test cycle on the OrangeHRM demo application (orangehrm.com). Test the Leave Management module (apply leave, approve leave, cancel leave). Log 3 real defects and track them through the defect life cycle.
45. Using the following data, calculate all 8 test metrics from section 8.3: Sprint had 100 planned TCs. Executed: 94. Passed: 89. Failed: 3. Blocked: 2. 11 defects logged in testing. 2 defects found in UAT. 1 defect found in production after release.
46. Write a daily status update for Day 6 of a 10-day sprint: 80/120 test cases executed, 74 passed, 4 failed, 2 blocked, 6 new defects (1 P1, 3 P2, 2 P3), 2 defects in Retest.
47. Write a one-page Test Summary Report for a sprint where: 120 TCs planned, 115 executed, 110 passed, 3 failed (all P3, deferred), 2 blocked (P4 deferred). 14 defects total: all resolved. Pass rate 95.7%. UAT signed off. Recommend Go or No-Go with justification.
48. Calculate DRE for a release where 45 defects were found in testing, 3 were found in UAT, and 1 was found in production. Interpret the result.

Quick Quiz

1. What does a Test Pass Rate of 85% at the end of a sprint tell you? Is this a Go or No-Go?

2. A test case is marked Blocked. What are your next steps as a tester?
3. What is the Defect Removal Efficiency formula and what does a score of 91% mean?
4. What sections must be included in a Test Summary Report?
5. How do you calculate Test Coverage and what does 100% coverage actually guarantee?

Interview Questions

1. Walk me through how you manage and report test progress in an Agile sprint.
2. What metrics do you track daily during test execution and why?
3. Describe a Test Summary Report you have written. What did it contain and who was the audience?
4. How do you make a release recommendation when there are still open defects?
5. What does a high Blocked Test Rate indicate and how do you address it?

Chapter Summary

• Test execution is a managed process with discipline and documentation at every step.
• Every test case needs a recorded result. Every failure needs a linked defect. No exceptions.
• Blocked test cases must be tracked, escalated, and resolved — not silently skipped.
• The eight core test metrics (execution rate, pass rate, DDR, DRE, etc.) tell the complete story of quality.
• Daily status updates keep stakeholders informed and prevent surprises at release time.
• The Test Summary Report is your professional closing statement. It should be clear, evidence-based, and actionable.

Capstone Project: Complete Level 1 QA Engagement

↑ Back to Index

This capstone simulates a real Level 1 QA engagement on a demo e-commerce application. You will perform every activity from the STLC using the techniques learned in this phase.

Application Under Test

Use the OpenCart demo application available at demo.opencart.com. Your scope is: User Registration, User Login, Product Search, Add to Cart, Checkout Process.

Deliverables

49. Requirement Review Report: Review the 10 requirements in the capstone requirements document. Identify testability issues and raise queries for each.
50. Test Plan: Write a 2-page test plan covering scope, strategy, schedule, resources, entry/exit criteria, and risk.
51. RTM: Create a complete RTM mapping all 10 requirements to test cases.
52. Test Cases: Write 50 test cases using at least 3 design techniques (BVA, EP, Decision Table, State Transition).
53. Test Execution: Execute all 50 test cases on the demo application. Record all results.
54. Defect Reports: Log every failed test case as a professional defect report in a tracking tool of your choice.
55. Triage Record: Conduct a mock triage on your defects. Document triage decisions for each defect.
56. Test Summary Report: Write a complete Test Summary Report with all metrics and a release recommendation.

Deliverable	Marks	Assessment Criteria
Requirement Review Report	10	Completeness of review, quality of queries raised
Test Plan	15	Completeness of sections, risk identification, realistic estimates
RTM	10	100% requirement coverage, correct linkage
Test Cases	20	Technique application, completeness, positive and negative cases
Test Execution	15	All cases executed and recorded, environment documented

Deliverable	Marks	Assessment Criteria
Defect Reports	15	Professional format, all fields complete, reproducible steps
Test Summary Report	15	Accurate metrics, clear recommendation, professional format

Case Study: The Production Defect That Changed Everything

↑ Back to Index

Background: Ravi was a Junior QA Engineer at a fintech startup. He joined 6 months after the product launched and found a team under constant pressure to release fast. Test cases existed but were not regularly updated. The RTM had not been touched in 4 months. Triage meetings were skipped. Defects were verbally communicated rather than logged.

The Incident

A production defect in the money transfer module allowed users to transfer funds using an expired OTP (one-time password). The OTP validation code had been refactored by a developer but the regression test suite had never been updated to cover OTP expiry scenarios. The requirement for OTP expiry was in the specification document but had never been mapped to a test case. The RTM had a gap.

The defect went live and was exploited by a small number of users before it was detected via transaction monitoring. The company faced a regulatory inquiry, customer compensation costs, and a 2-week emergency freeze on all releases while a full audit was conducted.

What Went Wrong

- The RTM had not been maintained. Refactored code changed behaviour but the test cases were not updated to reflect the changed OTP validation flow.
- Regression testing did not cover the OTP expiry scenario. This requirement existed in the spec but had no corresponding test case — a textbook example of an orphan requirement.
- Defects were not logged formally. Several developers had noted OTP-related issues in code comments but never raised them to QA.
- Triage meetings had been stopped to save time. Without triage, no one had visibility of risk accumulation.

What Ravi Did After the Incident

- Led a full RTM audit: mapped every requirement to its test case. Found 12 additional untested requirements.
- Implemented mandatory defect logging in JIRA for all issues, regardless of size.
- Restarted daily triage meetings with defined attendance and a 30-minute timebox.
- Introduced a bi-weekly regression suite review to keep test cases aligned with code changes.
- Wrote a Lessons Learned report and presented it to the CTO. Was promoted to Senior QA Engineer 6 months later.

Ravi's experience illustrates that the fundamentals taught in this phase — STLC discipline, RTM maintenance, defect logging, and triage — are not bureaucratic overhead. They are the professional infrastructure that prevents catastrophic failures. Every shortcut taken in QA process creates risk. The cost of the incident far exceeded the time that would have been saved by maintaining the RTM.

Mock Assessment: Phase 2 — Level 1 Junior QA Engineer

↑ Back to Index

Section A: Multiple Choice (20 marks)

57. Which STLC phase involves writing test cases and preparing test data? (a) Requirement Analysis (b) Test Planning (c) Test Case Development (d) Test Execution
58. BVA for a field accepting values 1 to 100 would test which values? (a) 1, 50, 100 (b) 0, 1, 2, 99, 100, 101 (c) 1, 100 (d) 50, 75, 100
59. A defect is marked as Critical Severity but P3 Priority. Which scenario does this best describe? (a) Login button does not work on a rarely-used admin panel (b) Typo on the homepage (c) Payment fails for all users (d) Report shows wrong data for the CEO
60. Which testing type is performed immediately after a specific bug fix is deployed? (a) Regression Testing (b) Smoke Testing (c) Sanity Testing (d) UAT
61. An RTM maps: (a) Test cases to defects (b) Requirements to test cases (c) Test cases to environments (d) Defects to requirements

Section B: Short Answer (30 marks)

62. Explain the difference between Entry Criteria and Exit Criteria in the STLC. Provide an example of each. (6 marks)
63. What are the six characteristics of a good requirement? Give one example of a requirement that violates the testability characteristic. (6 marks)
64. Describe the Equivalence Partitioning technique and apply it to a discount code field that accepts codes of 6-12 alphanumeric characters. (6 marks)
65. What is the Defect Removal Efficiency (DRE) metric? If 80 defects were found in testing, 5 in UAT, and 2 in production, what is the DRE? Is this acceptable? (6 marks)
66. List the 5 most important fields in a defect report and explain why each is critical. (6 marks)

Section C: Scenario-Based (50 marks)

67. You are a Junior QA Engineer assigned to test a hotel booking feature. The requirements state: 'Users can search for hotels by city, check-in date, check-out date, and number of guests. Results should show within 3 seconds. A maximum of 50 results should be displayed.' Using at least 3 test case design techniques, write 15 test cases for this feature. Include both positive and negative cases. (25 marks)
68. At the end of Sprint 8, you have the following data: 80 TCs planned, 75 executed, 68 passed, 5 failed (2 P2 deferred, 3 P3 deferred), 2 blocked (environment issue, both P4). 12 defects logged during sprint: 0 P1, 4 P2 (2 fixed, 2 deferred), 8 P3 (all fixed). UAT completed with 2 minor observations (no blockers). Write a Test Summary Report and make a Go/No-Go recommendation with full justification. (25 marks)

Quick Reference: Level 1 Junior QA Cheat Sheet

↑ Back to Index

STLC Phases

Phase	Key Deliverable	Entry	Exit
1. Requirement Analysis	RTM skeleton, Queries list	Signed-off requirements	Test conditions identified
2. Test Planning	Test Plan document	RTM available	Test Plan approved
3. Test Case Development	Test Cases, Test Data	Test Plan signed off	Test cases reviewed
4. Environment Setup	Env Setup Report	Test Cases ready	Environment validated
5. Test Execution	Execution Report, Defects	Environment stable	All TCs executed
6. Test Cycle Closure	Test Summary Report	All TCs executed	Report approved

Test Case Design Techniques

Technique	Use When	Key Rule
BVA	Numeric or date ranges	Test min-1, min, min+1, max-1, max, max+1
EP	Large input categories	One test per partition is sufficient
Decision Table	Multiple conditions, multiple outcomes	Each column = one test case
State Transition	Systems with states and events	Test all valid transitions and key invalid ones
Pairwise	Many parameters and values	Every pair of values tested at least once
Error Guessing	Experience-based risk areas	Empty, null, max length, special chars, duplicates

Defect Severity vs Priority

Severity Level	Meaning	Priority Level	Meaning
Critical / Blocker	System crash, data loss, security breach	P1	Fix immediately, blocks release

Severity Level	Meaning	Priority Level	Meaning
Major / High	Key feature broken, no workaround	P2	Fix in current sprint
Minor / Medium	Feature partially broken, workaround exists	P3	Fix in next sprint
Trivial / Low	Cosmetic issue, no functional impact	P4	Fix when time permits

Key Metrics Formulas

Metric	Formula	Target
Test Pass Rate	Passed / Executed x 100	> 95%
Test Execution Rate	Executed / Planned x 100	> 90%
Defect Detection Rate	Pre-release / (Pre + Post) x 100	> 95%
Defect Removal Efficiency	Pre-release / (Pre + Post) x 100	> 95%
Defect Rejection Rate	Rejected / Total Logged x 100	< 10%
Defect Leakage	Production defects / Total defects x 100	< 1%

Release Go/No-Go Quick Reference

Condition	Go Threshold	No-Go Trigger
P1 defects open	Zero	Any P1 open
P2 defects open	3 or fewer with plan	More than 3 without mitigation
Pass Rate	Above 95%	Below 90%
Execution Rate	Above 90%	Below 80%
UAT Sign-off	Formal sign-off received	Not signed off